1. Flow diagram:

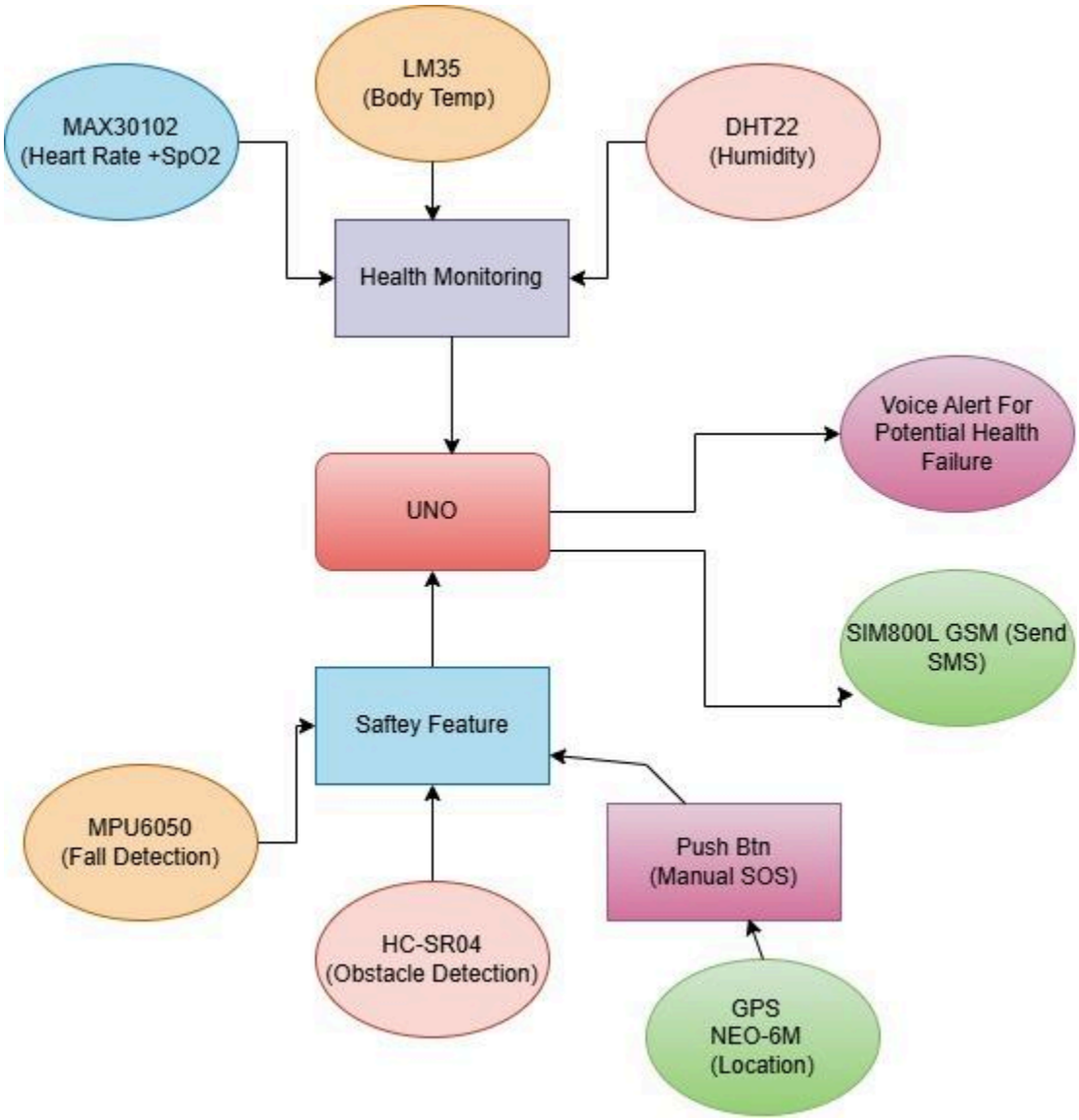

Fig : FLow diagram

The smart helmet integrates health monitoring, environmental sensing and safety/event handling around Arduino UNO. The Arduino UNO continuously fuses signals from physiological sensors (MAX30102, LM35), climate sensors (DHT22), and motion/proximity sensors (MPU6050, HC-SR04). Based on embedded rules, it issues local preventive alerts (voice) and, when necessary, sends text messages with location

using SIM800L (GSM) + NEO-6M (GPS). A manual SOS button gives the rider an immediate way to request help.

1.1 Health Monitoring Subsystem

- MAX30102 (HR): optical sensing of heart rate to detect unusual heart rate.
- LM35 (Body temperature): measures body temperature; if it stays high in hot weather, the system can warn of heat-stress risk.
- DHT22 (Ambient temp & humidity): enables computation of heat stress indicators (e.g., Heat Index).

The three sensors send data for Health Monitoring.

1.2 Safety Feature Subsystem

- MPU6050 (accelerometer + gyroscope): detects impact spikes and orientation changes characteristic of a fall/accident.
- HC-SR04 (ultrasonic): short-range proximity warning to reduce collision risk in dense traffic/slow maneuvers.
- Push Button (manual SOS): one-press emergency alert the rider can trigger instantly.

1.3 Communications & Notification

- Voice Alert (speaker/DFPlayer or equivalent): Alerts biker when sensor readings indicate a high risk of heatstroke.
- NEO-6M (GPS): provides latitude/longitude for emergency messages.
- SIM800L (GSM): transmits SMS to predefined contacts (automatic after confirmed fall or on manual SOS).

2. System architecture

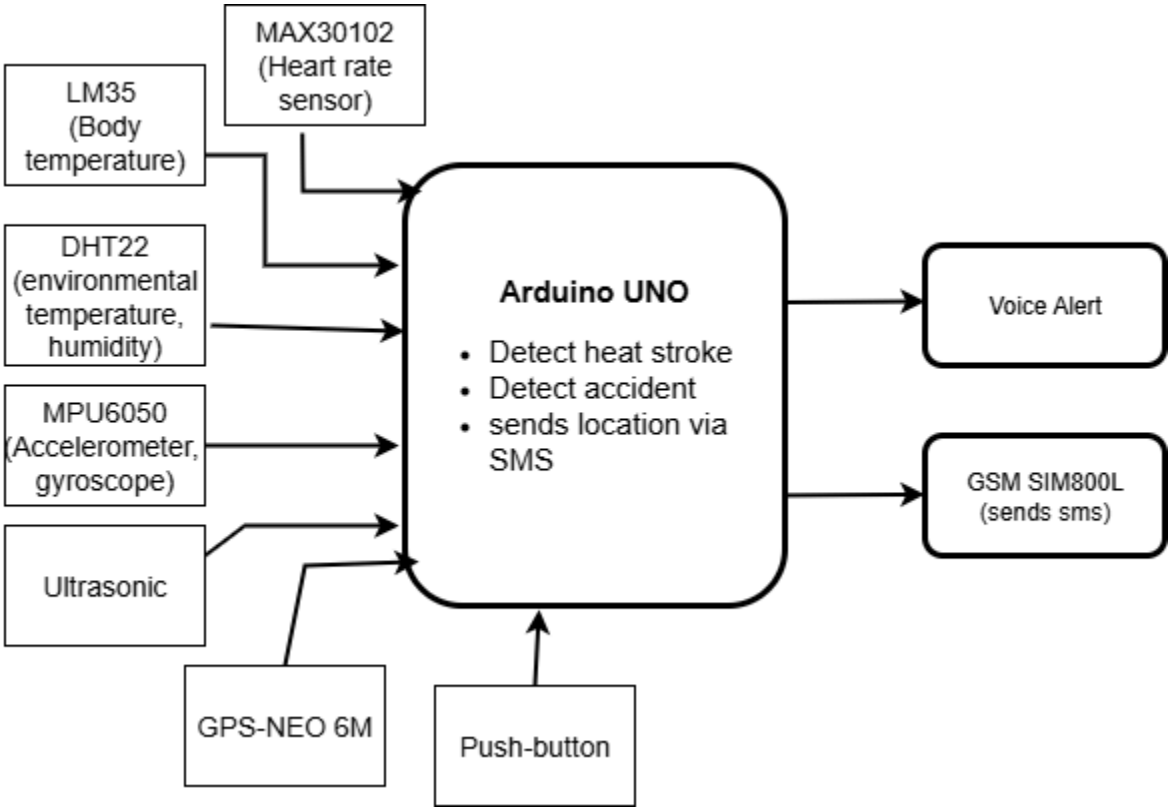

Fig : System architecture

An Arduino UNO acts as the decision hub. It fuses vital signs, climate and motion data. Arduino UNO to

(i) prevent incidents with on-helmet voice prompts

(ii) respond to crashes by sending an SOS SMS with GPS via GSM.

2.1. All sensor inputs feed into the Arduino UNO

- LM35 (Body temperature): rider temperature trend.
- DHT22 (Ambient temperature & humidity): climate context for Heat Index.
- MAX30102 (Heart rate): pulse/HR trend for fatigue/overheat cues.
- MPU6050 (Accelerometer + gyroscope): impact and orientation change for fall/accident detection.
- Ultrasonic (HC-SR04): short-range obstacle/proximity cue.

- GPS NEO-6M: latitude/longitude for SOS messages.

- Push-button: manual SOS trigger.

2.2. The Arduino UNO generates all system outputs

- Voice Alert (speaker/DFPlayer): Alerts the biker when sensor readings indicate a high risk of heatstroke.
- GSM SIM800L: sends SMS to preset contacts; includes GPS link.

3. Circuit diagram

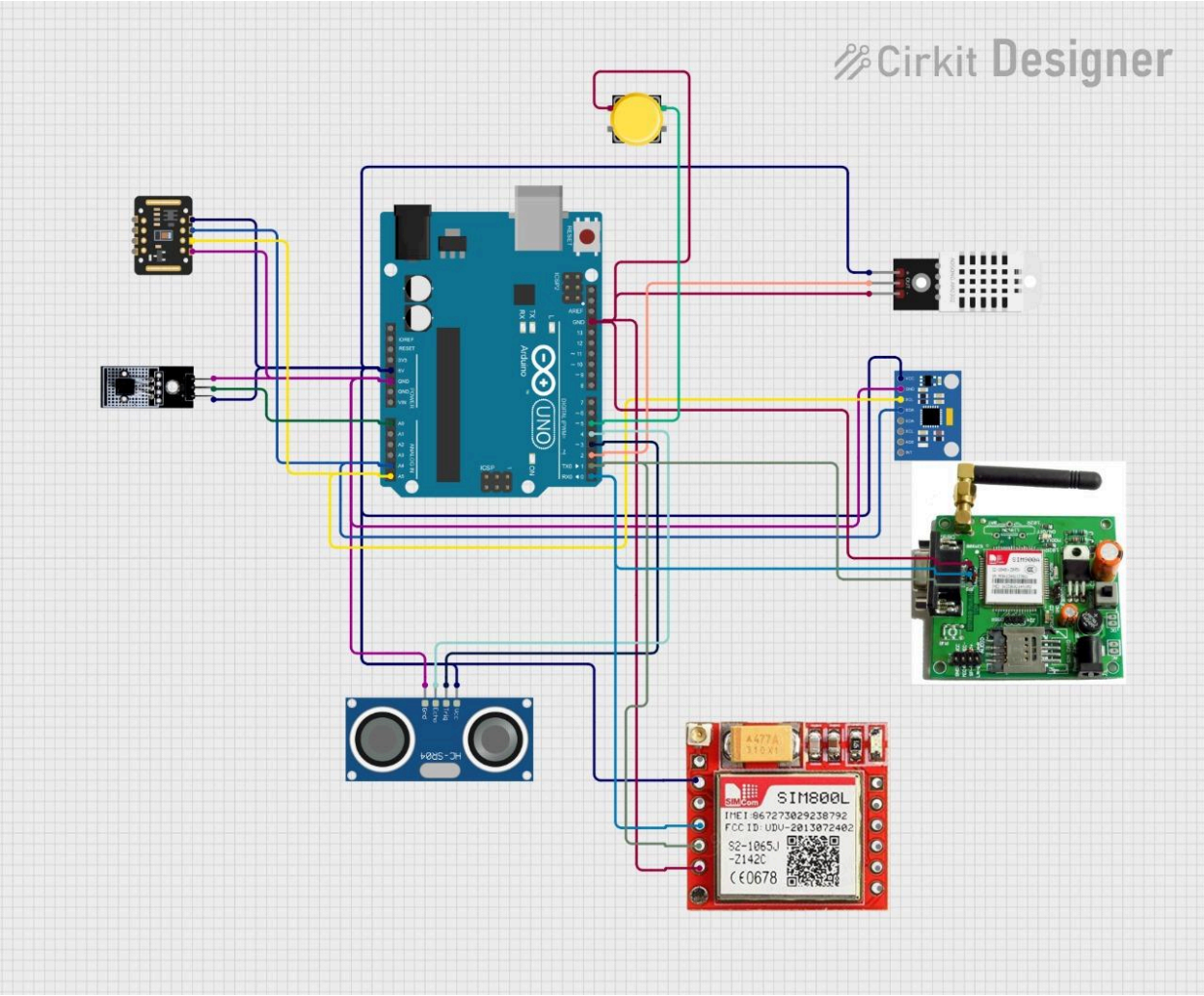

Fig: Circuit diagram

Arduino UNO is the hub. It reads health/climate/motion sensors, then:

- gives local warnings by voice/buzzer, and
- sends remote SOS by GSM (SIM800L) with GPS location.

3.1 Connections:

- I²C bus (shared):
  - MPU6050 (fall/impact) → SDA=A4, SCL=A5
  - MAX30102 (heart rate) → SDA=A4, SCL=A5
- Analog:
  - LM35 (body temp) → A1
- Digital sensors/IO:
  - DHT22 (ambient temp/humidity) → D4
  - HC-SR04 (ultrasonic) → TRIG=D8, ECHO=D9
  - Push button (manual SOS) → D7 (use INPUT_PULLUP)
- UART (serial):
  - SIM800L GSM → UNO D10=TX→SIM800 RX (via divider/level shift), D11=RX←SIM800 TX (SoftwareSerial)
  - NEO-6M GPS → UNO D5=RX, D6=TX (SoftwareSerial)
- Power & ground (all modules share GND):
  - Most sensors: 5 V
  - SIM800L: requires a stable ~4.0–4.2 V
  - GPS: requires 5V.

3.2 Work of each part :

- MAX30102 + LM35 + DHT22 → health/climate readings for heat-stress detection.
- MPU6050 → detects falls (impact + orientation change).
- HC-SR04 → proximity warning in slow traffic.
- Push button → rider can trigger SOS immediately.
- GPS (NEO-6M) → latitude/longitude for alerts.
- SIM800L → sends SMS with the GPS link to emergency contacts.

4. Pseudocode of the implemented logics and code

4.1. Constants & Globals

- Pins:
  - DHT → D4,
  - Ultrasonic → TRIG D8 / ECHO D9,
  - Vibration SW-420 → D12,
  - Heart sensor → A0,
  - SIM800L UART → D10 (TX→SIM RX), D11 (RX←SIM TX)

- Phone number = "+8801893072703"
- Fall threshold (from MPU6050) = 15000 (accel² units)
- Heartbeat sampling interval = 2 ms (≈500 Hz)
- Refractory period between beats = 250 ms
- Rolling IBI window size = 5
- Variables: rawVal, lpFiltered, baseline, threshOffset, aboveThreshold, lastBeatMs, ibiMs, window[], bpm, haveBpm, fallDetected, timers

4.2. `HeartbeatSampleAndDetect()`

```
if (time since lastSample < 2 ms) return

lastSample = now

rawVal = read analog HEART_PIN

lpFiltered = 0.9*lpFiltered + 0.1*rawVal          // low-pass

baseline  = 0.999*baseline + 0.001*lpFiltered      // slow drift track

threshold = baseline + threshOffset                // dynamic threshold

isAbove = (lpFiltered > threshold)

dt = now - lastBeatMs

if (isAbove AND NOT aboveThreshold AND dt > REFRACTORY_MS):

   // rising edge -> beat

   if (lastBeatMs exists):
```

```
      ibiMs = dt
      if (300 ≤ ibiMs ≤ 2000):                    // 30..200 BPM guard
         put ibiMs into circular window
         avgIbi = average(non-zero values in window)
         bpm = round(60000 / avgIbi)
         haveBpm = true
   lastBeatMs = now
aboveThreshold = isAbove
```

4.3. ReadDistanceCM()

Send 10 µs pulse on TRIG

Measure ECHO high time with timeout

if duration == 0 → return -1

else return (duration * 0.034 / 2) // microseconds → centimeters

4.4. MakeCall()

print "ATD<phone>;" to SIM800L

wait ~20 s

print "ATH" to hang up

4.5.SendSMSAlert(temp, hum, dist)

send "AT+CMGF=1"   // text mode

send 'AT+CMGS="<phone>"'

compose lines:

  "ALERT: Fall detected! Immediate help required."

  "BPM: <bpm or N/A>"

```
  "Temp: <temp C>"
  "Hum: <hum %>"
  "Dist: <dist cm>"
send Ctrl+Z (0x1A) to transmit
```

4.6. setup()

```
Start Serial(9600), Wire(I²C)
Initialize MPU6050; stop if connection fails
Start DHT22
Configure pins: TRIG=OUTPUT, ECHO=INPUT, VIBRATION=INPUT, HEART=INPUT
Start SIM800L serial(9600); send "AT" to check
Print "Setup complete"
```

4.7. loop()

```
HeartbeatSampleAndDetect()  // keep near 500 Hz via internal timer

// Fast tasks (every ~50 ms): fall detection
if (50 ms elapsed):
   read ax, ay, az from MPU6050
   totalAccSq = ax² + ay² + az²
   mpuFall = (totalAccSq < fallThreshold)         // free-fall cue
   vibrationDetected = (SW-420 pin is HIGH)

   if (mpuFall AND vibrationDetected AND NOT fallDetected):
      fallDetected = true
      temp = DHT temperature
```

```
        hum  = DHT humidity
        dist = ReadDistanceCM()
        MakeCall()
        SendSMSAlert(temp, hum, dist)
    else if (NOT (mpuFall AND vibrationDetected)):
        fallDetected = false

// Slow diagnostics (every 1 s): print status
if (1 s elapsed):
    read accel, temp, hum, dist, vibration
    print all readings + heart raw value + BPM (or '--')
```

5. References

[1] M. Manjesh and S. S. Sudarshan, “Smart Helmet Using GSM & GPS Technology for Accident Detection and Reporting System,” *International Journal of Research*, vol. 1, no. 10, pp. 68–70, 2014.

[2] Amulya **et al.**, “Smart Helmet With Message Alert System,” *International Journal of Scientific Research in Science and Technology (IJSRST)*, vol. 4, no. 5, pp. 133–137, 2018.

[3] M. M. Mehbub, T. Rahman, and F. Ahmed, “Real-time Accident Detection and Physiological Signal Monitoring to Enhance Motorbike Safety and Emergency Response,” *arXiv preprint* arXiv:2403.19085, 2024.

[4] K. Silawarawet, P. Kaewchukul, and S. Saadprai, “Heat Stroke Warning System Prototype for Athletes: A Pilot Study,” *Sensors*, vol. 25, Art. no. 294, Jan. 2025, doi: 10.3390/s25020294.

[5] M. E. Ahmad **et al.**, “Internet of Things-Based Smart Helmet with Accident Identification and Logistics Monitoring for Delivery Riders,” *Proceedings*, vol. 58, no. 1, 2024.